User Manual

for MPC560XB PWM MCAL Driver

Document Number: UM13PWMASR4.0R1.0.1
Rev 1.2

freescale™

Contents

Section number **Title** **Page**

Chapter 1
Revision History

Chapter 2
Introduction

Chapter 3
Driver

Chapter 4
Tresos Configuration Plug-in

Chapter 1
Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	17/02/2012	Subramanya M Naik	Changes for MPC560XB 0.9.0 BETA
1.1	17/05/2013	Manoj S R	Changes for MPC560XB 1.0.0 RTM
1.2	06/06/2014	Quyen Trong	Changes for MPC560XB 1.0.1 RTM

Chapter 2
Introduction

This User Manual describes AUTOSAR Pulse Width Modulation (PWM) forMPC560XB.

AUTOSAR PWM driver configuration parameters and deviations from the specification are described in Pwm Driver chapter of this document. AUTOSAR PWM driver requirements and APIs are described in the AUTOSAR PWM driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor :

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100

Table 2-1. MPC560XB Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTOmotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
GPIO	General Purpose Input Output
GPT	General Purpose Timer
MCAL	Microntroller Abstraction Layer
MIDE	Multi Integrated Development Environment
PWM	Pulse Width Modulation
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

Title	Version
AUTOSAR PWM Driver Software Specification Document	V2.5.0
MPC5607B Reference Manual	Rev. 7.2, May 2012
MPC5604B Reference Manual	Rev. 8.2, September 2013
5606B Reference Manual	Rev. 2, May 2014
5602D Reference Manual	Rev. 4.2, Dec 2013
Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3
Driver

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003Pwm Driver Software Specification document (See Reference List).

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003PWM Driver Software Specification document (See Table **Reference List**).

3.2 Driver Design Summary

The AUTOSAR PWM Driver Specification defines the functionality, API and the configuration of the AUTOSAR Basic Software module PWM Driver. Each PWM channel is linked to hardware PWM which belongs to the microcontroller. EMIOS unified channels on EMIOS_A & EMIOS_B are used to implement the PWM functionality.

The driver provides services for initialization and control of the microcontroller internal PWM stage (pulse width modulation). The PWM module generates pulses with variable pulse width. It allows the selection of the duty cycle and the signal period time.

The modes supported:

- OPWFMB
- OPWMB
- OPWMT
- DAOC
- OPWMCB

The channels supporting OPWFMB modes (with Variable Period and Variable Duty Cycle and having bus Internal Counter) are

- Ch0, Ch8, Ch16, Ch23, Ch24 on EMIOS_A
- Ch0, Ch8, Ch16, Ch23, Ch24 on EMIOS_B

The channels supporting OPWMB and OPWMT modes (with Fixed Period and Variable Duty Cycle and having bus Counter BusA or Bus Diverse) are

- Ch0 - Ch31 on EMIOS_A and EMIOS_B

The channels supporting DAOC modes (with Variable Period and Variable Duty Cycle and having bus Internal Counter or Counter BusA or Bus Diverse) are

- Ch1 - Ch7 and Ch9 - Ch15 on EMIOS_A
- Ch9 - Ch15 on EMIOS_B

The channels supporting OPWMCB modes (with Variable Period and Variable Duty Cycle and having bus Counter BusA or Bus Diverse) are

- Ch1, Ch2, Ch3, Ch4, Ch5, Ch6, Ch7 on EMIOS_A

3.3 Calling Driver APIs

1. void Pwm_GetVersionInfo (Std_VersionInfoType* versioninfo)

This function returns the version information of this module.

versioninfo - Pointer where to store the version information of this module.

```
typedef struct
{
    uint16  vendorID;
    uint16  moduleID;
    uint8   sw_major_version;
    uint8   sw_minor_version;
    uint8   sw_patch_version;
} Std_VersionInfoType;
```

Sample code:

```
/* main_app.c */
.....................
Std_VersionInfoType version;
uint16  vendor_id;

Pwm_Init( Pwm_Cfg1);
#if (PWM_VERSION_INFO_API == STD_ON)
    Pwm_GetVersionInfo( &version);
    vendor_id = version.vendorID;
#endif /* (PWM_VERSION_INFO_API == STD_ON) */
.....................
```

Note

This API can be used only if PWM_VERSION_INFO_API is set to STD_ON.

2. void Pwm_Init (const Pwm_ConfigType* ConfigPtr)

This function initializes the module.

ConfigPtr - Pointer to driver configuration. This is the type of the external data structure containing the overall initialization data for the PWM driver. Furthermore it contains pointers to controller configuration structures. The contents of the initialization data structure are PWM hardware specific.

For Pre-Compile configuration the Pwm_cfg.h file will contain the extern declaration of "Pwm_ConfigType" structure.

```
.....................
#if (PWM_PRECOMPILE_SUPPORT == STD_ON )
    #define PWM_INIT_CONFIG_PC_DEFINE \
        extern CONST(Pwm_ConfigType, PWM_CONST) Pwm_InitConfig_PC;
#endif
.....................
```

Pwm_Cfg.c file will contains the structure type:

```
.....................
CONST(Pwm_ConfigType, PWM_CONST) Pwm_InitConfig_PC = {
.....
};
.....................
```

For Post-Build configuration the Pwm_Cfg.h file will contain the extern declaration of "Pwm_ConfigType" structure.

```
.....................
#if (PWM_PRECOMPILE_SUPPORT == STD_OFF )
    #define PWM_INIT_CONFIG_PB_DEFINES \
        extern CONST(Pwm_ConfigType, PWM_CONST) PwmChannelConfigSet_0;
    #endif
.....................
```

Pwm_PBcfg.c file will contains the structure type:

```
.....................
CONST(Pwm_ConfigType, PWM_CONST) PwmChannelConfigSet_0 = {
    .....
};
.....................
```

Sample code:

```
/* main_app.c */
.....................
#if (PWM_PRECOMPILE_SUPPORT == STD_ON )
    CONST(Pwm_ConfigType, PWM_CONST) *Pwm_Cfg1 = &Pwm_InitConfig_PC;
#else
    CONST(Pwm_ConfigType, PWM_CONST) *Pwm_Cfg1 = &PwmChannelConfigSet_0;
#endif
Pwm_Init( Pwm_Cfg1);
.....................
```

3. void Pwm_DeInit(void)

The function **Pwm_DeInit** will de-initialize the PWM module.

The call to **Pwm_DeInit** will set the state of the PWM output signals to the idle state. Also it will disable PWM interrupts and PWM signal edge notifications.

The function is compile time configurable on/off by the parameter PwmDeInitApi{PWM_DE_INIT_API}

```
#if (PWM_DE_INIT_API == STD_ON)
    void Pwm_DeInit(void);
#endif
```

Sample code

```
/* main_app.c */
.............................
Pwm_Init(Pwm_Cfg);
.............................
#if (PWM_DE_INIT_API == STD_ON)
    Pwm_DeInit();
#endif
.............................
```

4. void Pwm_SetDutyCycle(Pwm_ChannelType ChannelNumber, uint16 DutyCycle)

The function **Pwm_SetDutyCycle** will set the duty cycle of the PWM channel.

If the requested duty cycle is 0% or 100% the output will be set to PWM_HIGH or PWM_LOW depending on the configured polarity and the duty cycle

Before a call to **Pwm_SetDutyCycle** the driver must be initialized by a prior call to **Pwm_Init()**

If the PwmDevErorDetect switch is enabled, the function will check it's parametters and will act according to PWM_SWS

The function is compile time configurable on/off by the parameter PwmSetDutyCycle{PWM_SET_DUTY_CYCLE_API}

```
#if (PWM_SET_DUTY_CYCLE_API == STD_ON)
    void Pwm_SetDutyCycle(
        Pwm_ChannelType ChannelNumber,
        uint16 DutyCycle
    );
#endif
```

Sample code

```
/* main_app.c */
.............................
Pwm_Init( Pwm_Cfg );
.............................
uint16 duty = 0x4000; /* valid duty cycle */
Pwm_ChannelType channel = 1;  /* valid channel number */
.............................
Pwm_SetDutyCycle( channel, duty );
.............................
Pwm_DeInit();
.............................
```

5. void Pwm_SetPeriodAndDuty(Pwm_ChannelType ChannelNumber, Pwm_PeriodType Period, uint16 DutyCycle)

The function **Pwm_SetPeriodAndDuty** shall set the period and the duty cycle of a PWM channel.

If the requested duty cycle is 0% or 100% the output will be set to PWM_HIGH or PWM_LOW depending on the configured polarity and the duty cycle

If the period is set to zero the setting of the duty-cycle is not relevant. In this case the output will be zero (zero percent duty-cycle).

Before a call to **Pwm_SetPeriodAndDuty** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmSetPeriodAndDuty{PWM_SET_PERIOD_AND_DUTY_API}

```
#if (PWM_SET_PERIOD_AND_DUTY_API == STD_ON)
    void Pwm_SetPeriodAndDuty(
        Pwm_ChannelType ChannelNumber,
        Pwm_PeriodType Period,
        uint16 DutyCycle
    );
#endif
```

Sample code

```
/* main_app.c */
.............................
Pwm_Init(Pwm_Cfg);
.............................
```

```
uint16 duty = 0x4000; /* valid duty cycle */
Pwm_PeriodType Period = 0x1000; /* valid period */
Pwm_ChannelType channel = 1;  /* valid channel number */
.............................
Pwm_SetPeriodAndDuty( ChannelNumber, Period, DutyCycle );
.............................
Pwm_DeInit();
.............................
```

6. void Pwm_SetOutputToIdle(void)

A call to **Pwm_SetOutputToIdle** will set immediately the PWM output to the configured Idle state.

Before a call to **Pwm_SetOutputToIdle** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable by the parameter PwmSetOutputToIdle{PWM_SET_OUTPUT_TO_IDLE_API}

After the call of the function **Pwm_SetOutputToIdle** variable period type channels shall be reactivated using the Api Pwm_SetPeriodAndDuty( ) to activate the PWM channel with the new passed period.

After the call of the function **Pwm_SetOutputToIdle**, channels shall be reactivated using the Api Pwm_SetDutyCycle( ) to activate the PWM channel with the old period.

The function is compile time configurable on/off by the parameter PwmSetOutputToIdle{PWM_SET_OUTPUT_TO_IDLE_API}

```
#if (PWM_SET_OUTPUT_TO_IDLE_API == STD_ON)
    void Pwm_SetOutputToIdle(
        Pwm_ChannelType ChannelNumber
    );
#endif
```

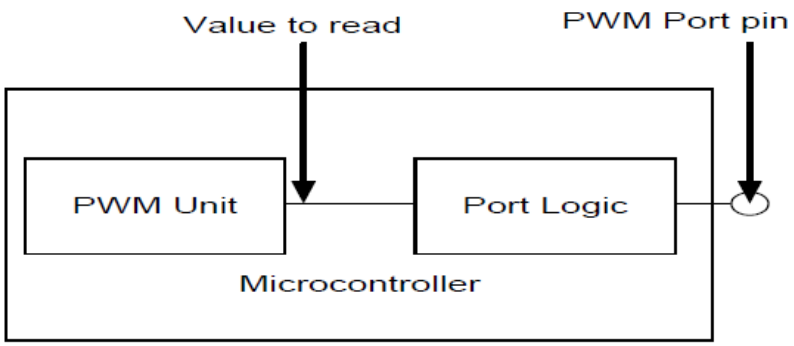
Sample code

```
/* main_app.c */
/* presume the channel nr. 1 is configured as PWM_VARIABILE_PERIOD */
.............................
Pwm_Init(Pwm_Cfg);
.............................
uint16 duty = 0x4000; /* valid duty cycle */
Pwm_PeriodType Period = 0x1000; /* valid period */
Pwm_ChannelType channel = 1;  /* valid channel number */
.............................
Pwm_SetOutputToIdle(channel); /* disable the channel */
.............................
Pwm_SetPeriodAndDuty( ChannelNumber, Period, DutyCycle );  /* enable the channel
with new period and duty*/
.............................
Pwm_SetOutputToIdle(channel); /* disable the channel */
.............................
Pwm_SetDutyCycle( channel, duty ); /* re-enable the channel with new duty */
```

```
.....................................
```

7. Pwm_OutputStateType Pwm_GetOutputState(Pwm_ChannelType ChannelNumber)

The function **Pwm_GetOutputState** will read the internal state of the PWM output signal and return it as defined in the diagram below



Before a call to **Pwm_GetOutputState** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter Pwm_GetOutputState{PWM_GET_OUTPUT_STATE_API}

```
#if (PWM_GET_OUTPUT_STATE_API == STD_ON)
    Pwm_OutputStateType Pwm_GetOutputState(
        Pwm_ChannelType ChannelNumber
    );
#endif
```

Sample code

```
/* main_app.c */
.....................................
Pwm_Init(Pwm_Cfg);
.....................................
Pwm_OutputStateType outState;
Pwm_ChannelType channel;
.....................................
outState = Pwm_GetOutputState( channel ); /* outState will always be PWM_LOW, see
the note below */
.....................................
```

8. void Pwm_EnableNotification(Pwm_ChannelType ChannelNumber, Pwm_EdgeNotificationType Notification)

The function **Pwm_EnableNotification will** enable the PWM signal edge notification according to notification parameter.

A call to **Pwm_EnableNotification** will cancel pending interrupts

Before a call to **Pwm_EnableNotification** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmNotificationSupported {PWM_NOTIFICATION_SUPPORTED}

```
#if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
    void Pwm_EnableNotification(
        Pwm_ChannelType ChannelNumber,
        Pwm_EdgeNotificationType Notification
    );
#endif
```

Sample code

```
/* main_app.c */
.............................
Pwm_Init(Pwm_Cfg);
.............................
Pwm_ChannelType channel;
Pwm_EdgeNotificationType notification = PWM_RISING_EDGE;
.............................
Pwm_EnableNotification(  channel, notification);
.............................
```

9. void Pwm_DisableNotification(Pwm_ChannelType ChannelNumber)

The function Pwm_DisableNotification will disable the PWM signal edge notification.

Before a call to **Pwm_DisableNotification** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmNotificationSupported {PWM_NOTIFICATION_SUPPORTED}

```
#if (PWM_NOTIFICATION_SUPPORTED == STD_ON)
    void Pwm_DisableNotification(
        Pwm_ChannelType ChannelNumber
    );
#endif
```

Sample code

```
/* main_app.c */
.............................
Pwm_Init(Pwm_Cfg);
.............................
Pwm_ChannelType channel;
Pwm_EdgeNotificationType notification = PWM_RISING_EDGE;
.............................
Pwm_EnableNotification(  channel, notification);
```

```
.............................
Pwm_DisableNotification( channel );
.............................
```

Non-AUTOSAR function

10.void Pwm_SetCounterBus(Pwm_ChannelType ChannelNumber, uint32 Bus)

The function will change the frequency of pwm channels running on PWM_MODE_OPWMB or PWM_MODE_OPWMT mode.

Before a call to **Pwm_SetCounterBus** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmSetCounterBusApi {PWM_SETCOUNTERBUS_API}

```
#if (PWM_SETCOUNTERBUS_API == STD_ON)
    FUNC (void, PWM_CODE) Pwm_SetCounterBus(
        VAR(Pwm_ChannelType, AUTOMATIC) ChannelNumber,
        VAR(uint32, AUTOMATIC) Bus
    );
#endif
```

11.void Pwm_SetClockMode(Pwm_SelectPrescalerType Prescaler)

This function is useful to set the prescalers that divide the PWM channels clock frequency.

Before a call to **Pwm_SetClockMode** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmEnableDualClockMode {PWM_DUAL_CLOCK_MODE}

```
#if (PWM_DUAL_CLOCK_MODE==STD_ON)
    FUNC (void, PWM_CODE) Pwm_SetClockMode(
        VAR(Pwm_SelectPrescalerType, AUTOMATIC) Prescaler
    );
#endif
```

12.voidPwm_SetChannelOutput(Pwm_ChannelType ChannelNumber, Pwm_StateType State)

This function is useful to set the state of the PWM pin as requested for the current cycle and continues with normal PWM operation from the next cycle.

Before a call to **Pwm_SetChannelOutput** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmSetChannelOutputApi {PWM_SETCHANNELOUTPUT_API}

```
#if (PWM_SETCHANNELOUTPUT_API == STD_ON)
    FUNC (void, PWM_CODE) Pwm_SetChannelOutput(
        VAR(Pwm_ChannelType, AUTOMATIC) ChannelNumber,
        VAR(Pwm_StateType, AUTOMATIC) State
    );
#endif
```

13. void Pwm_GetChannelState(Pwm_ChannelType ChannelNumber)

This function return the DutyCycle value of the channel.

Before a call to **Pwm_GetChannelOutput** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmGetChannelOutputApi {PWM_GETCHANNELSTATE_API}

```
#if (PWM_GETCHANNELSTATE_AP == STD_ON)
    FUNC (Pwm_DutyType, PWM_CODE) Pwm_GetChannelState(
        VAR(Pwm_ChannelType, AUTOMATIC) ChannelNumber
                );
#endif
```

14. void Pwm_BufferTransferEnableDisable(uint8 ModuleIndex, uint32 ChannelMask)

This function is useful to enable/disable the buffer transfer to synchronize multiple PWM channels.

Before a call to **Pwm_BufferTransferEnableDisable** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmBufferTransferEnableDisableApi {PWM_BUFFERTRANSFERENABLEDISABLE_API}

```
    #if (MULTI_PWM_CHANNEL_SYNCH == STD_ON)
extern FUNC (void, PWM_CODE) Pwm_BufferTransferEnableDisable
                        (
                            VAR(uint8, AUTOMATIC) ModuleIndex,
                            VAR(uint32, AUTOMATIC) ChannelMask
                        );
    #endif
```

15. void Pwm_SetTriggerdelay(Pwm_ChannelType ChannelNumber, uint16 TriggerDelay)

This function is useful to set the trigger delay to opwmt mode.

Before a call to **Pwm_SetTriggerdelay** the driver must be initialized by a prior call to **Pwm_Init()**

The function is compile time configurable on/off by the parameter PwmSetTriggerdelayApi {PWM_SETTRIGGERDELAY_API}

```
    #if (PWM_SETTRIGGERDELAY_API == STD_ON)
FUNC (void, PWM_CODE) Pwm_SetTriggerDelay
                      (
                            VAR(Pwm_ChannelType, AUTOMATIC) ChannelNumber,
                            VAR(uint16, AUTOMATIC) TriggerDelay
                      )
    #endif
```

3.4 Additional Requirements and Deviations

The channels supporting OPWMB and OPWMT modes (with Fixed Period and Variable Duty Cycle and having bus Counter BusA or Bus Diverse) are • Ch0 - Ch31 on EMIOS_A and EMIOS_B.

The channels supporting DAOC mode (with Variable Period and Variable Duty Cycle and having bus Internal Counter or Counter BusA or Bus Diverse) are • Ch1 - Ch7 and Ch9 - Ch15 on EMIOS_A • Ch9 - Ch15 on EMIOS_B.

The channels supporting OPWMCB mode (with Variable Period and Variable Duty Cycle and having bus Counter BusA or Bus Diverse) are • Ch1 - Ch7 on EMIOS_A .

The driver deviates from the AUTOSAR PWM Driver software specification in some places

The deviations from the AUTOSAR PWM Driver software specification are listed in table 2-3.

Table 3-1. Deviations Status Column Description

Term	Definition
N/A	Not Applicable
N/V	Not Verifiable
N/S	Out of Scope
N/R	Unclear Requirement
N/F	Not Fully Implemented

Table continues on the next page...

Table 3-1. Deviations Status Column Description (continued)

Term	Definition
N/I	Not Implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the PWM driver.

Table 3-2. PWM Deviations Table

Requirement	Status	Description	Notes
PWM070	N/S	All time units used within the API services of the PWM driver shall be of the unit ticks	Upper layer responsibility to pass ticks to PWM driver API's
eMIOS Implementation specific		The period of the eMIOS unified channel used as reference for OPWMT channels must be <= 0xFFFE or the driver will not work correctly.	User responsibility to configure the reference channel used for OPWMT with period <= 0xFFFE.

3.4.1 Pwm Center Aligned Feature (OPWMCB mode)

a. The below list of HW resources that are consumed for center aligned implementation:

- 2 eMIOS Channels using up and down modulus counter buffered.

- 1 eMIOS Channels using classic modulus counter per trigger/ADC needed.

- 1 eMIOS Channel for each modulus counter bus used (up, up-down).

- 1 CTU Channel per trigger/ADC needed.

- ADC channel(s).

b. Guideline to configure the counter buses using MCAL for the selected modes (MCB up/down counter and MCB up counter)::

- In Pwm_Init() MCB up/down counter shall be initialized, selecting the channel configured as counter bus by the OPWMCB channels (BUS_A or BUS_DIVERSE). The period value for MCB up/down counter bus shall be taken from the OPWMCB channels configuration. Please note that this counter bus channel shall not be reconfigured or used for any other purpose!

- MCB up counter shall be configured as an OPWFMB PWM channel.

c. Guideline on sequence of PWM APIs to be called to adjust the OPWMT channels at runtime when the period of the OPWMCB channels is changed::

- To adjust trigger and offset value of the OPWMT mode channel Pwm_SetDutyCycle() function shall be called after some delay i.e. one period delay, because after changing the period value of the OPWMCB mode channel the trigger value and offset will not be adjusted automatically.

d. Guideline on how to start the 2 counter bus channels at the same time (tick-level sync) using the MCAL::

- Initialize MCU driver by disabling the eMIOS "GlobalPrescalerEnable" parameter from the "McueMIOSSettings" container.

- Initialize PWM driver.

- Reinitialize MCU driver by enabling the eMIOS "GlobalPrescalerEnable" parameter from the "McueMIOSSettings" container or call directly the dedicated MCU non-AUTOSAR API for GPREN manipulation.

e. Guideline to change duty cycle value synchronously for both OPWMCB and OPWMT channels::

1.Disable transfer of buffered register values for a list of EMIOS channels Pwm_BufferTransferEnableDisable (EmiosModuleIndex, ChannelMask)/*1 bit per channel */

2.Call Pwm_SetDutyCycle() to update duty cycle value for all the channels(OPWMCB and OPWMT)*/

3.Enable transfer of buffered register values for a list of EMIOS channels Pwm_BufferTransferEnableDisable (EmiosModuleIndex, ChannelMask)/*1 bit per channel */

f. Guideline to change period and duty cycle value synchronously for both OPWMCB and OPWMT channels::

1.Disable transfer of buffered register values for a list of EMIOS channels Pwm_BufferTransferEnableDisable (EmiosModuleIndex, ChannelMask)/*1 bit per channel */

2.Call Pwm_SetPeriodAndDuty () to update period and duty cycle value for all the channels OPWMCB and OPWFMB(Counter bus of trigger channel))

3.Enable transfer of buffered register values for a list of EMIOS channels Pwm_BufferTransferEnableDisable (EmiosModuleIndex, ChannelMask)/*1 bit per channel */

4.User application shall wait for current period to complete User_Application_Delay (till end of current period);

5.Call Pwm_SetDutyCycle () for trigger channel (OPWMT) to adjust trigger delay and offset for new period changed in Step2.

Note: To use this Pwm_BufferTransferEnableDisable () API the switch MULTI_PWM_CHANNEL_SYNCH should be enabled in PWM general container.:

Limitations::

1.Not all the EMIOS channels are buffered (Ex: OPWMT offset and trigger delay values).

2. PwmChangeRegisterA parameter shall be disabled.

3. Changing period and duty cycle (Pwm_SetPeriodAndDuty ()) will affect the synchronization between OPWMCB and OPWMT channels.

4. Changing the duty cycle value (Pwm_SetDutyCycle ()) from 100% to 0% will not work always, same Pwm_SetDutyCycle () API shall be called more than once to achieve 0% duty cycle, because of the below line mentioned in reference manual:

i. “If A1 is greater than the maximum value of the selected counter bus period, then a 0% duty cycle is produced, only if the pin starts the current cycle in the opposite of EDPOL value.”

ii. Polarity bit (EDPOL) will not be changed, to avoid glitches in the PWM waveform.

3.5 API Reference

APIs of all functions supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

3.5.1 Function Pwm_Init

This function initializes the Pwm driver.

Prototype: `void Pwm_Init(const Pwm_ConfigType *ConfigPtr);`

Table 3-3. Pwm_Init Arguments

Type	Name	Direction	Description
const `Pwm_ConfigType` *	ConfigPtr	**input**	pointer to PWM top configuration structure

Return: void

The function Pwm_Init shall initialize all internals variables and the used PWM structure of the microcontroller according to the parameters specified in ConfigPtr. If the duty cycle parameter equals:

- 0% or 100% : Then the PWM output signal shall be in the state according to the configured polarity parameter;
- >0% and <100%: Then the PWM output signal shall be modulated according to parameters period, duty cycle and configured polarity.

The function Pwm_SetDutyCycle shall update the duty cycle always at the end of the period if supported by the implementation and configured with PwmDutycycleUpdatedEndperiod.

The driver shall avoid spikes on the PWM output signal when updating the PWM period and duty.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The function Pwm_Init shall disable all notifications. The reason is that the users of these notifications may not be ready. They can call Pwm_EnableNotification to start notifications.

The function Pwm_Init shall only initialize the configured resources and shall not touch resources that are not configured in the configuration file.

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection is enabled, calling the routine Pwm_Init while the PWM driver and hardware are already initialized will cause a development error PWM_E_ALREADY_INITIALIZED. The desired functionality shall be left without any action.

For pre-compile and link time configuration variants, a NULL pointer shall be passed to the initialization routine. In this case the check for this NULL pointer has to be omitted.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.2 Function Pwm_DeInit

This function deinitializes the Pwm driver.

Prototype: `void Pwm_DeInit(void);`

Return: void

The function Pwm_DeInit shall deinitialize the PWM module.

The function Pwm_DeInit shall set the state of the PWM output signals to the idle state. The function Pwm_DeInit shall disable PWM interrupts and PWM signal edge notifications. The function Pwm_DeInit shall be pre-compile time configurable On/Off by the configuration parameter PwmDeInitApi function prototype. If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.3 Function Pwm_SetDutyCycle

This function sets the dutycycle for the specified Pwm channel.

Prototype: `void Pwm_SetDutyCycle(Pwm_ChannelType ChannelNumber, uint16 DutyCycle);`

Table 3-4. Pwm_SetDutyCycle Arguments

Type	Name	Direction	Description
Pwm_ChannelType	ChannelNumber	**input**	pwm channel id
uint16	DutyCycle	**input**	pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Return: void

The function Pwm_SetDutyCycle shall set the duty cycle of the PWM channel.

The function Pwm_SetDutyCycle shall set the PWM output state according to the configured polarity parameter, when the duty cycle = 0% or 100%. The function Pwm_SetDutyCycle shall modulate the PWM output signal according to parameters period, duty cycle and configured polarity, when the duty cycle > 0 % and < 100%.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

Note

MISRA 2004 Required Rule 10.1, Implicit conversion changes signedness

3.5.4 Function Pwm_SetPeriodAndDuty

This function sets the period and the dutycycle for the specified Pwm channel.

Prototype: `void Pwm_SetPeriodAndDuty(Pwm_ChannelType ChannelNumber, Pwm_PeriodType Period, uint16 DutyCycle);`

Table 3-5. Pwm_SetPeriodAndDuty Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id
`Pwm_PeriodType`	Period	**input**	- pwm signal period value
`uint16`	DutyCycle	**input**	- pwm dutycycle value 0x0000 for 0% ... 0x8000 for 100%

Return: void

The function Pwm_SetPeriodAndDuty shall set the duty cycle of the PWM channel.

If development error detection for the Pwm module is enabled, the PWM functions shall check the channel class type and raise development error PWM_E_PERIOD_UNCHANGEABLE if the PWM channel is not declared as a variable period type.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

The Pwm module shall comply with the following scaling scheme for the duty cycle:

- 0x0000 means 0%.
- 0x8000 means 100%.
- 0x8000 gives the highest resolution while allowing 100% duty cycle to be represented with a 16 bit value. As an implementation guide, the following source code example is given: AbsoluteDutyCycle = ((uint32)AbsolutePeriodTime * RelativeDutyCycle) >> 15;

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.5 Function Pwm_SetOutputToIdle

This function sets the generated pwm signal to the idle value configured.

Prototype: `void Pwm_SetOutputToIdle(Pwm_ChannelType ChannelNumber);`

Table 3-6. Pwm_SetOutputToIdle Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id

Return: void

The function Pwm_SetOutputToIdle shall set immediately the PWM output to the configured Idle state.

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

After the call of the function Pwm_SetOutputToIdle, variable period type channels shall be reactivated either using the Api `Pwm_SetPeriodAndDuty()` to activate the PWM channel with the new passed period or Api `Pwm_SetDutyCycle()` to activate the PWM channel with the old period.

After the call of the function Pwm_SetOutputToIdle, fixed period type channels shall be reactivated using only the API Api `Pwm_SetDutyCycle()` to activate the PWM channel with the old period.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.6 Function Pwm_GetOutputState

This function returns the signal output state.

Prototype: `Pwm_OutputStateType Pwm_GetOutputState(Pwm_ChannelType ChannelNumber);`

Table 3-7. Pwm_GetOutputState Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id

Return: Pwm_OutputStateType pwm signal output logic value

Table 3-8. Pwm_GetOutputState Returns

Value	Description
PWM_LOW	- The output state of PWM channel is low
PWM_HIGH	- The output state of PWM channel is high

The function Pwm_GetOutputState shall read the internal state of the PWM output signal and return it as defined in the diagram below (see PWM_SWS).

If development error detection for the Pwm module is enabled, the PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

Due to real time constraint and setting of the PWM channel (project dependant), the output state can be modified just after the call of the service Pwm_GetOutputState.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

Note

Due to hardware limitation this function will always return PWM_LOW

3.5.7 Function Pwm_DisableNotification

This function disables the user notifications.

Prototype: `void Pwm_DisableNotification(Pwm_ChannelType ChannelNumber);`

Table 3-9. Pwm_DisableNotification Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id

Return: void

If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

All functions from the PWM module except Pwm_Init, Pwm_DeInit and Pwm_GetVersionInfo shall be re-entrant for different PWM channel numbers. In order to keep a simple module implementation, no check of PWM088 must be performed by the module. The function Pwm_DisableNotification shall be pre compile time configurable On/Off by the configuration parameter: PwmNotificationSupported.

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.8 Function Pwm_EnableNotification

This function enables the user notifications.

Prototype: `void Pwm_EnableNotification(Pwm_ChannelType ChannelNumber, Pwm_EdgeNotificationType Notification);`

Table 3-10. Pwm_EnableNotification Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id
`Pwm_EdgeNotificationType`	Notification	**input**	- notification type to be enabled

Return: void

The function Pwm_EnableNotification shall enable the PWM signal edge notification according to notification parameter. If development error detection for the Pwm module is enabled:

- The PWM functions shall check the parameter ChannelNumber and raise development error PWM_E_PARAM_CHANNEL if the parameter ChannelNumber is invalid.

If development error detection for the Pwm module is enabled, when a development error occurs, the corresponding PWM function shall:

- Report the error to the Development Error Tracer.
- Skip the desired functionality in order to avoid any corruptions of data or hardware registers (this means leave the function without any actions).
- Return pwm level low for the function Pwm_GetOutputState.

If the PwmDevErorDetect switch is enabled, API parameter checking is enabled. The detailed description of the detected errors can be found in chapter Error classification and chapter API specification (see PWM_SWS).

If development error detection for the Pwm module is enabled, if any function (except Pwm_Init) is called before Pwm_Init has been called, the called function shall raise development error PWM_E_UNINIT.

3.5.9 Function Pwm_GetVersionInfo

This function returns Pwm driver version details.

Prototype: `void Pwm_GetVersionInfo(Std_VersionInfoType *versioninfo);`

Table 3-11. Pwm_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	**input, output**	- pointer to Std_VersionInfoType output variable

Return: void

The function Pwm_GetVersionInfo shall return the version information of this module. The version information includes: Module Id, Vendor Id, Vendor specific version number.

3.5.10 Function Pwm_SetClockMode

Implementation specific function to change the peripheral clock frequency.

Prototype: `void Pwm_SetClockMode(Pwm_SelectPrescalerType Prescaler);`

Table 3-12. Pwm_SetClockMode Arguments

Type	Name	Direction	Description
`Pwm_SelectPrescalerType`	Prescaler	**input**	- prescaler type

This function is useful to set the prescalers that divide the PWM channels clock frequency.

3.5.11 Function Pwm_SetCounterBus

function to change the frequency of pwm channels running on PWM_MODE_OPWMB or PWM_MODE_OPWMT mode.

Prototype: `void Pwm_SetCounterBus(Pwm_ChannelType ChannelNumber, uint32 Bus);`

Table 3-13. Pwm_SetCounterBus Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id
uint32	Bus	**input**	- the eMIOS bus to change to

This function is useful to change the frequency of the output PWM signal between two counter buses frequency

3.5.12 Function Pwm_SetChannelOutput

function to set the state of the PWM pin as requested for the current cycle

Prototype: `void Pwm_SetChannelOutput(Pwm_ChannelType ChannelNumber, Pwm_StateType State);`

Table 3-14. Pwm_SetChannelOutput Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id
`Pwm_StateType`	State	**input**	- Active/Inactive state of the channel

This function is useful to set the state of the PWM pin as requested for the current cycle and continues with normal PWM operation from the next cycle

3.5.13 Function Pwm_GetChannelState

Implementation specific function to return the dutycycle value

Prototype: `void Pwm_GetChannelState(Pwm_ChannelType ChannelNumber);`

Table 3-15. Pwm_GetChannelState Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id

This function return the DutyCycle value of the channel.

3.5.14 Function Pwm_BufferTransferEnableDisable

Implementation specific function to enable/disable the buffer transfer.

Prototype: `void Pwm_BufferTransferEnableDisable( uint8 ModuleIndex, uint32 ChannelMask);`

Table 3-16. Pwm_BufferTransferEnableDisable Arguments

Type	Name	Direction	Description
`uint8`	ModuleIndex	**input**	- eMIOS0, eMIOS1 selection
`uint32`	ChannelMask	**input**	- selection of eMIOS ModuleIndex channals

This function is useful to enable/disable the buffer transfer to synchronize multiple PWM channels.

3.5.15 Function Pwm_SetTriggerdelay

Implementation specific function to change the trigger delay

Prototype: `void Pwm_SetTriggerdelay( Pwm_ChannelType ChannelNumber, uint16 TriggerDelay);`

Table 3-17. Pwm_SetTriggerdelay Arguments

Type	Name	Direction	Description
`Pwm_ChannelType`	ChannelNumber	**input**	- pwm channel id
`uint16`	TriggerDelay	**input**	- pwm trigger delay

This function is useful to set the trigger delay to opwmt mode.

3.6 Structure Definitions

Data structures supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

3.6.1 Structure Pwm_ChannelConfigType

pwm channel high level configuration structure

Defines the class of PWM channel Pwm_ChannelClassValue - channel type: Variable/ Fixed period Pwm_Polarity - Pwm signal polarity: High or low Pwm_DefaultPeriodValue - Default value for period Pwm_DefaultDutyCycleValue - Default value for duty cycle: [0-0x8000] (0-100%) Pwm_IdleState - Pwm signal idle state: High or low Pwm_Channel_Notification - Pointer to notification function IpType - the IP used to implement this specific Pwm channel SpecificCfg - Pwm channel IP specific parameters

Declaration

```
typedef struct
{
  const Pwm_IpType IpType,
  const Pwm_NotifyType Pwm_Channel_Notification,
  const Pwm_ChannelClassType Pwm_ChannelClassValue,
  const uint16 Pwm_DefaultDutyCycleValue,
  const Pwm_PeriodType Pwm_DefaultPeriodValue,
  const Pwm_OutputStateType Pwm_IdleState,
  const Pwm_OutputStateType Pwm_Polarity,
  const Pwm_LLD_ChannelConfigType SpecificCfg
} Pwm_ChannelConfigType;
```

Table 3-18. Structure Pwm_ChannelConfigType member description

Member	Description
IpType	the IP used to implement this specific Pwm channel
Pwm_Channel_Notification	Pointer to notification function.
Pwm_ChannelClassValue	channel type: Variable/Fixed period
Pwm_DefaultDutyCycleValue	Default value for duty cycle: [0-0x8000] (0-100%)
Pwm_DefaultPeriodValue	Default value for period.
Pwm_IdleState	Pwm signal idle state: High or low.
Pwm_Polarity	Pwm signal polarity: High or low.
SpecificCfg	Pwm channel IP specific parameters.

3.6.2 Structure Pwm_ConfigType

pwm high level configuration structure

This is the type of data structure containing the initialization data for the PWM driver. ChannelCount - number of configured channels ChannelsPtr - pointer to the configured channels

Declaration

```
typedef struct
{
  const Pwm_ChannelType ChannelCount,
  const Pwm_ChannelConfigType * ChannelsPtr
} Pwm_ConfigType;
```

Table 3-19. Structure Pwm_ConfigType member description

Member	Description
ChannelCount	number of configured channels
ChannelsPtr	pointer to the configured channels

3.6.3 Structure Pwm_EMIOS_ChannelConfigType

EMIOS IP specific channel configuration structure for the PWM functionality.

Declaration

```
typedef struct
{
    CONST(Pwm_HwChannelType, PWM_CONST) Pwm_HwChannelID;
    CONST(uint32,      PWM_CONST)               Pwm_Emios_ModuleAddr;
    CONST(uint32,      PWM_CONST)               Pwm_Emios_ChannelAddr;
    CONST(Pwm_EmiosCtrlParamType, PWM_CONST) Pwm_ParamValue;
#if (PWM_DUAL_CLOCK_MODE == STD_ON)
    CONST(uint32, PWM_CONST) Pwm_Prescaler_Alternate;
#endif
    CONST(Pwm_PeriodType, PWM_CONST) Pwm_Offset;

#ifdef PWM_FEATURE_OPWMT
    #if (PWM_FEATURE_OPWMT == STD_ON)
    CONST(Pwm_PeriodType, PWM_CONST) Pwm_TriggerDelay;
    #endif
#endif
    CONST(boolean, PWM_CONST) Pwm_OffsetTriggerDelayAdjust;
#ifdef PWM_FEATURE_OPWMCB
    #if (PWM_FEATURE_OPWMCB == STD_ON)
    CONST(uint16, PWM_CONST) Pwm_DeadTime;
    #endif
#endif
} Pwm_EMIOS_ChannelConfigType;
```

Table 3-20. Structure Pwm_EMIOS_ChannelConfigType member description

Member	Description
Pwm_HwChannelID	eMIOS HW channel id
Pwm_Offset	leading edge of the output pulse
Pwm_ParamValue	EMIOSC[n] control.
Pwm_Prescaler_Alternate	prescaler value
Pwm_DeadTime	dead time

3.6.4 Structure Pwm_LLD_ChannelConfigType

Low level channel configuration structure.

Pwm channel eMIOS specific configuration structure

Declaration

```
typedef struct
{
  const Pwm_EMIOS_ChannelConfigType EmiosCfg
} Pwm_LLD_ChannelConfigType;
```

Table 3-21. Structure Pwm_LLD_ChannelConfigType member description

Member	Description
EmiosCfg	Pwm channel eMIOS specific configuration structure.

3.6.5 Structure Std_VersionInfoType

This type shall be used to request the version of a BSW module using the "ModuleName"_GetVersionInfo() function.

Declaration

```
typedef struct
{
  uint16 moduleID,
  uint8 sw_major_version,
  uint8 sw_minor_version,
  uint8 sw_patch_version,
  uint16 vendorID
} Std_VersionInfoType;
```

Table 3-22. Structure Std_VersionInfoType member description

Member	Description
moduleID	BSW module ID.
sw_major_version	BSW module software major version.
sw_minor_version	BSW module software minor version.
sw_patch_version	BSW module software patch version.
vendorID	vendor ID

3.7 Define Definitions

Data defines supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

3.7.1 Define PWM_DE_INIT_API

Switch to indicate that Pwm_DeInit API is supported.

Definition:
```
#define PWM_DE_INIT_API (STD_ON)
```

3.7.2 Define PWM_DEV_ERROR_DETECT

Switch for enabling the development error detection.

Definition:
```
#define PWM_DEV_ERROR_DETECT (STD_ON)
```

3.7.3 Define PWM_DUAL_CLOCK_MODE

Switch to indicate that Pwm_SetClockMode API is supported.

Definition:
```
#define PWM_DUAL_CLOCK_MODE (STD_OFF)
```

3.7.4 Define PWM_DUTY_CYCLE_100

100% dutycycle

Definition:
```
#define PWM_DUTY_CYCLE_100 ((uint16)0x8000U)
```

3.7.5 Define PWM_DUTY_PERIOD_UPDATED_ENDPERIOD

Switch for enabling the update of the period parameter at the end of the current period.

Definition:
```
#define PWM_DUTY_PERIOD_UPDATED_ENDPERIOD (STD_ON)
```

3.7.6 Define PWM_DUTYCYCLE_UPDATED_ENDPERIOD

Switch for enabling the update of the duty cycle parameter at the end of the current period.

Definition:
```
#define PWM_DUTYCYCLE_UPDATED_ENDPERIOD (STD_ON)
```

3.7.7 Define PWM_E_ALREADY_INITIALIZED

API Pwm_Init service called while the PWM driver has already been initialised.

Definition:`#define PWM_E_ALREADY_INITIALIZED 0x14U`

3.7.8 Define PWM_E_COUNTERBUS

Error code generated when Pwm_SetCounterBus is called with an invalid parameter.

Definition:`#define PWM_E_COUNTERBUS 0x19U`

3.7.9 Define PWM_E_DUTYCYCLE_RANGE

Pwm_SetDutyCycle or Pwm_SetPeriodAndDuty called with invalid dutycycle range.

Definition:`#define PWM_E_DUTYCYCLE_RANGE 0x17U`

3.7.10 Define PWM_E_PARAM_CHANNEL

API service used with an invalid channel Identifier.

Definition:`#define PWM_E_PARAM_CHANNEL 0x12U`

3.7.11 Define PWM_E_PARAM_CONFIG

API Pwm_Init service called with wrong parameter.

Definition:`#define PWM_E_PARAM_CONFIG 0x10U`

3.7.12 Define PWM_E_PARAM_NOTIFICATION

Invalid polarity selected for edge notification.

Definition:`#define PWM_E_PARAM_NOTIFICATION 0x18U`

3.7.13 Define PWM_E_PARAM_NOTIFICATION_NULL

NULL function is configured as notification callback.

Definition:`#define PWM_E_PARAM_NOTIFICATION_NULL 0x16U`

3.7.14 Define PWM_E_PARAM_POINTER

Generated when a NULL pointer is passed to Pwm_GetVersionInfo function.

Definition:`#define PWM_E_PARAM_POINTER 0x15U`

3.7.15 Define PWM_E_PERIOD_UNCHANGEABLE

Usage of unauthorized PWM service on PWM channel configured a fixed period.

Definition:`#define PWM_E_PERIOD_UNCHANGEABLE 0x13U`

3.7.16 Define PWM_E_UNINIT

API service used without module initialization.

Definition:`#define PWM_E_UNINIT 0x11U`

3.7.17 Define PWM_GET_OUTPUT_STATE_API

Switch to indicate that Pwm_GetOutputState API is supported.

Definition:`#define PWM_GET_OUTPUT_STATE_API (STD_ON)`

3.7.18 Define PWM_INIT_ID

API service ID of Pwm_Init function.

Definition:`#define PWM_INIT_ID 0x00U`

3.7.19 Define PWM_NOTIFICATION_SUPPORTED

Switch to indicate that the notifications are supported.

Definition:`#define PWM_NOTIFICATION_SUPPORTED (STD_ON)`

3.7.20 Define PWM_PRECOMPILE_SUPPORT

Definition:
```
#define PWM_PRECOMPILE_SUPPORT STD_OFF
```

3.7.21 Define PWM_SET_DUTY_CYCLE_API

Switch to indicate that Pwm_SetDutyCycle API is supported.

Definition:
```
#define PWM_SET_DUTY_CYCLE_API (STD_ON)
```

3.7.22 Define PWM_SET_OUTPUT_TO_IDLE_API

Switch to indicate that Pwm_SetOutputToIdle API is supported.

Definition:
```
#define PWM_SET_OUTPUT_TO_IDLE_API (STD_ON)
```

3.7.23 Define PWM_SET_PERIOD_AND_DUTY_API

Switch to indicate that Pwm_SetPeriodAndDuty API is supported.

Definition:
```
#define PWM_SET_PERIOD_AND_DUTY_API (STD_ON)
```

3.7.24 Define PWM_VERSION_INFO_API

Switch to indicate that Pwm_GetVersionInfo API is supported.

Definition:
```
#define PWM_VERSION_INFO_API (STD_ON)
```

3.7.25 Define PWM_DEINIT_ID

API service ID of Pwm_DeInit function.

Definition:
```
#define PWM_DEINIT_ID 0x01U
```

3.7.26 Define PWM_SETPERIODANDDUTY_ID

API service ID of Pwm_SetPeriodAndDuty function.

Definition:`#define PWM_SETPERIODANDDUTY_ID 0x03U`

3.7.27 Define PWM_SETOUTPUTTOIDLE_ID

API service ID of Pwm_SetOutputToIdle function.

Definition:`#define PWM_SETOUTPUTTOIDLE_ID 0x04U`

3.7.28 Define PWM_GETOUTPUTSTATE_ID

API service ID of Pwm_GetOutputState function.

Definition:`#define PWM_GETOUTPUTSTATE_ID 0x05U`

3.7.29 Define PWM_DISABLENOTIFICATION_ID

API service ID of Pwm_DisableNotification function.

Definition:`#define PWM_DISABLENOTIFICATION_ID 0x06U`

3.7.30 Define PWM_ENABLENOTIFICATION_ID

API service ID of Pwm_EnableNotification function.

Definition:`#define PWM_ENABLENOTIFICATION_ID 0x07U`

3.7.31 Define PWM_GETVERSIONINFO_ID

API service ID of Pwm_GetVersionInfo function.

Definition:`#define PWM_GETVERSIONINFO_ID 0x08U`

3.7.32 Define PWM_SCHM_PROTECTRESOURCE_ID

API service ID of Pwm_Schm_ProtectResource function.

Definition:`#define PWM_SCHM_PROTECTRESOURCE_ID 0x09`

3.7.33 Define PWM_SCHM_UNPROTECTRESOURCE_ID

API service ID of Pwm_Schm_UnprotectResource function.

Definition:
```
#define PWM_SCHM_UNPROTECTRESOURCE_ID 0x0A
```

3.7.34 Define MULTI_PWM_CHANNEL_SYNCH

Define is API service ID , Implementation specific used to enable/disable the buffer transfer.

Definition:
```
#define MULTI_PWM_CHANNEL_SYNCH (STD_ON)
```

3.7.35 Define PWM_DISABLE_DEM_REPORT_ERROR_STATUS

Switch to indicate that Dem_ReportErrorStatus API is supported

Definition:
```
#define PWM_DISABLE_DEM_REPORT_ERROR_STATUS STD_ON
```

3.7.36 Define PWM_GET_CHANNEL_STATE_API

Switch to indicate that PWM_GET_CHANNEL_STATE API is supported.

Definition:
```
#define PWM_GET_CHANNEL_STATE_API (STD_ON)
```

3.7.37 Define PWM_SET_TRIGGER_DELAY_API

Switch to indicate that Pwm_SetTriggerDelay API is supported.

Definition:
```
#define PWM_SET_TRIGGER_DELAY_API (STD_ON)
```

3.8 Enums Definitions

Enums defined by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

3.8.1 Enumeration Pwm_ChannelClassType

Defines class of PWM channel.

PWM_VARIABLE_PERIOD - The PWM channel has a variable period. The duty cycle and the period can be changed. PWM_FIXED_PERIOD - The PWM channel has a fixed period. Only the duty cycle can be changed. PWM_FIXED_PERIOD_SHIFTED - The PWM channel has a fixed shifted period. Impossible to change it (only if supported by hardware)

Table 3-23. Enumeration Pwm_ChannelClassType Values

Value	Description
PWM_VARIABLE_PERIOD = 0	
PWM_FIXED_PERIOD	
PWM_FIXED_PERIOD_SHIFTED	

3.8.2 Enumeration Pwm_EdgeNotificationType

Definition of the type of edge notification of a PWM channel.

PWM_RISING_EDGE - Notification will be called when a rising edge occurs on the PWM output signal. PWM_FALLING_EDGE - Notification will be called when a falling edge occurs on the PWM output signal. PWM_BOTH_EDGES - Notification will be called when either a rising edge or falling edge occur on the PWM output signal.

Table 3-24. Enumeration Pwm_EdgeNotificationType Values

Value	Description
PWM_RISING_EDGE = 0	
PWM_FALLING_EDGE	
PWM_BOTH_EDGES	

3.8.3 Enumeration Pwm_IpType

IP used to implement this channel.

This channel is implemented using one eMIOS HW unified channel

Table 3-25. Enumeration Pwm_IpType Values

Value	Description
PWM_EMIOS_CHANNEL = 0	

3.8.4 Enumeration Pwm_OutputStateType

Output state of a PWM channel.

PWM_HIGH - The output state of PWM channel is HIGH PWM_LOW - The output state of PWM channel LOW

Table 3-26. Enumeration Pwm_OutputStateType Values

Value	Description
PWM_LOW = 0	
PWM_HIGH	

3.8.5 Enumeration Pwm_SelectPrescalerType

Prescaler type.

Table 3-27. Enumeration Pwm_SelectPrescalerType Values

Value	Description
PWM_NORMAL = 0	
PWM_ALTERNATE	

3.8.6 Enumeration Pwm_StateType

Defines state of PWM channel.

PWM_ACTIVE - The PWM pin will be in the same state of configured polarity PWM_INACTIVE - The PWM pin will be in the opposite state of configured polarity

Table 3-28. Enumeration Pwm_StateType Values

Value	Description
PWM_ACTIVE = 0	
PWM_INACTIVE	

3.9 Typedef Definitions

Typedefs supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

3.9.1 Typedef Pwm_ChannelType

channel id typedef, used as type for both logical and hw channel ids.

Type: uint8

3.9.2 Typedef Pwm_EmiosCtrlParamType

eMIOS unified channel control register value

Type: uint32

3.9.3 Typedef Pwm_HwChannelType

EMIOS HW channel id type, make sure it is the same type as Pwm_ChannelType.

Type: uint8

3.9.4 Typedef Pwm_PeriodType

channel period typedef

Type: uint16

Chapter 4
Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the PWM Driver. The parameters are described below.

4.1 Config Variant

Table 4-1. IMPLEMENTATION_CONFIG_VARIANT

Description	Defines whether pre-compile version is used. Using this option with VariantPostBuild value, Tresos can generate many PwmChannelConfigSet variants.VariantPreCompile refers to precompile time configuration parameters. VariantPostBuild refers to a mix of precompile and postbuild time configuration parameters. If Config Variant = VariantPreCompile, the files Pwm_Cfg.h and Pwm_Cfg.c should be used. If Config Variant = VariantPostBuild, the files Pwm_Cfg.h and Pwm_PBcfg.c should be used.
Class	Implementation Specific Parameter
Range	VariantPreCompile, VariantPostBuild
Default	VariantPreCompile
Source File	Pwm_Cfg.h
Source Representation	#define PWM_PRECOMPILE_SUPPORT
Autosar 4.0 Requirement	NA
NOTE	*This parameter permit to generate many configurations if VariantPostBuild is selected.*

4.2 PwmConfigurationOfOptApiServices

This section will describe the API configuration options supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

4.2.1 PwmDeInitApi

Table 4-2. PwmDeInitApi

Description	Adds / removes the service Pwm_DeInit() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DE_INIT_API <STD_OFF, STD_ON>

4.2.2 PwmGetOutputState

Table 4-3. PwmGetOutputState

Description	Adds / removes the service Pwm_GetOutputState() from the code
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_GET_OUTPUT_STATE_API <STD_OFF, STD_ON>

4.2.3 PwmSetDutyCycle

Table 4-4. PwmSetDutyCycle

Description	Adds / removes the service Pwm_SetDutyCycle() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SET_DUTY_CYCLE_API <STD_OFF, STD_ON>

4.2.4 PwmSetOutputToIdle

Table 4-5. PwmSetOutputToIdle

Description	Adds / removes the service Pwm_SetOutputToIdle() from the code.
Class	Autosar Parameter

Table continues on the next page...

Table 4-5. PwmSetOutputToIdle (continued)

Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SET_OUTPUT_TO_IDLE_API <STD_OFF, STD_ON>

4.2.5 PwmSetPeriodAndDuty

Table 4-6. PwmSetPeriodAndDuty

Description	Adds / removes the service Pwm_SetPeriodAndDuty() from the code.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SET_PERIOD_AND_DUTY_API <STD_OFF, STD_ON>

4.2.6 PwmVersionInfoApi

Table 4-7. PwmVersionInfoApi

Description	Preprocessor switch to indicate that the Pwm_ GetVersionInfo is supported
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_VERSION_INFO_API <STD_OFF, STD_ON>

4.3 PwmGeneral

This section will describe the general configuration options supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

4.3.1 PwmDevErorDetect

Table 4-8. PwmDevErorDetect

Description	Preprocessor switch for enabling the development error detection
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DEV_ERROR_DETECT <STD_OFF, STD_ON>

4.3.2 PwmChangeRegisterA

Table 4-9. PwmChangeRegisterA

Description	Preprocessor switch to indicate if register B or register A are updated when a change in dutycycle for a pwm channel configured in OPWMT mode is requested.
Class	Non-Autosar Parameter
Range	True, False
Default	false
Source File	Pwm_Cfg.h
Source Representation	#define PWM_CHANGE_REGISTER_A_SWITCH <STD_OFF or STD_ON>
NOTE	

4.3.3 PwmDutycycleUpdatedEndperiod

Table 4-10. PwmDutycycleUpdatedEndperiod

Description	Switch for enabling the update of the duty cycle parameter at the end of the current period
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DUTYCYCLE_UPDATED_ENDPERIOD <STD_OFF, STD_ON>
NOTE	*PwmDutycycleUpdatedEndPeriod parameter is not used in driver code. Retained in configuration for consistency with AutosarEcuParamdef.xml rev 17. The EMIOS implementations of the PWM driver will always update the dutycycle at the end of the period where the update is made.*

4.3.4 PwmNotificationSupported

Table 4-11. PwmNotificationSupported

Description	Preprocessor switch to indicate that the notifications are supported.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_NOTIFICATION_SUPPORTED <STD_OFF, STD_ON>

4.3.5 PwmPeriodUpdatedEndperiod

Table 4-12. PwmPeriodUpdatedEndperiod

Description	Preprocessor switch for enabling the update of the period at the end of the current period
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DUTY_PERIOD_UPDATED_ENDPERIOD <STD_OFF, STD_ON>
NOTE	*PwmPeriodUpdatedEndperiod parameter is not implemented in driver code. Retained in configuration for consistency with AutosarEcuParamdef.xml rev 17. The EMIOS implementations of the PWM driver will always update the period at the end of the period where the update is made.*

4.3.6 PwmIndex

Table 4-13. PwmIndex

Description	Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.
Class	Autosar Parameter
Range	0 - 255
Default	0
Source File	N/A
Source Representation	N/A
NOTE	*PwmIndex parameter is not implemented in driver code. Retained in configuration for consistency with AutosarEcuParamdef.xml rev 17. The EMIOS implementation of the PWM driver is not using this parameter.*

4.3.7 PwmClockFromMCU

Table 4-14. PwmClockFromMCU

Description	Switch to enable/disable the clock reference from MCU plugin. The default value is True and will enable the PWM plugin to compute the EMIOS clock using information from the MCU plugin. If set to False it will disable the MCU clock reference and enable the PwmEmiosClockValue field. PwmEmiosClockValue can be used to inform the Pwm plugin about the value of the EMIOS clock value. It will NOT set the EMIOS clock to the specified value. This feature should be used for testing/debug purpose only. For production the EMIOS clock must be referenced from the MCU plugin.
Class	Implementation specific parameter
Range	True, False
Default	True
Source File	N/A
Source Representation	N/A
NOTE	

4.3.8 PwmCpuClockRef

Table 4-15. PwmCpuClockRef

Description	Reference to MCU configuration used. Enabled when PwmClockFromMcu is set.
Class	Implementation specific parameter
Range	Any valid reference to MCU configuration
Default	N/A
Source File	N/A
Source Representation	N/A
NOTE	*PwmClockRef parameter is not implemented in driver code. This reference is used to transform the period from seconds to ticks during plug-in generation*

4.3.9 PwmEmiosClockValue

Table 4-16. PwmEmiosClockValue

Description	Field used to inform the Pwm plugin about the value of the EMIOS clock value. It will NOT set the EMIOS clock to the specified value. It's enabled only when PwmClockFromMCU is set to false. Should be used only for testing/debug purpose only.
Class	Implementation specific parameter
Range	True, False
Default	N/A

Table continues on the next page...

Table 4-16. PwmEmiosClockValue (continued)

Source File	N/A
Source Representation	N/A
NOTE	

4.3.10 PwmGenerateClockTreeDebugInfo

Table 4-17. PwmGenerateClockTreeDebugInfo

Description	Switch to enable/disable the generation of debug information regarding the eMIOS Unified channel clock tree. Use this feature to debug problems regarding the period of pwm channels. Debug information will be available as comments in Pwm_Cfg.C and Pwm_PBCfg.c Example: /* --- Unified Channel clock debug information --- PwmPeriodDefaultUnits:Period_in_ticks PwmPeriodDefault:32767.0 this_unified_channel_period_in_ticks:32767 emios unified channel clock source: INTERNAL COUNTER this_unified_channel_prescaler_value:1 this_unified_channel_clock_frequency: 6.4E7 Hz this_unified_channel_clock_period:1.5625E-8 configured_channel_period: 5.11984375E-4 seconds configured_channel_freq:1953.1846064638205 Hz */
Class	Implementation specific parameter
Range	True, False
Default	False
Source File	N/A
Source Representation	N/A
NOTE	

4.4 PwmNonAUTOSAR

This section will describe the non-autosar configuration options supported by the driver .

4.4.1 PwmEnableDualClockMode

Table 4-18. PwmEnableDualClockMode

Description	Preprocessor switch that adds or removes the services Pwm_SetClockMode() from the code. This function is called when the prescaler value needs to be change to maintain same period at different frequency.
Class	Non-Autosar Parameter
Range	True, False
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DUAL_CLOCK_MODE <STD_OFF, STD_ON>

4.4.2 PwmSetChannelOutputApi

Table 4-19. PwmSetChannelOutputApi

Description	Preprocessor switch to indicate that the Pwm_SetChannelOutput is supported
Class	Non-Autosar Parameter
Range	True, false
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SETCHANNELOUTPUT_API <STD_OFF or STD_ON>
NOTE	

4.4.3 PwmSetCounterBusApi

Table 4-20. PwmSetCounterBusApi

Description	Preprocessor switch to indicate that the Pwm_SetCounterBus is supported.
Class	Non-Autosar Parameter
Range	True, false
Default	True
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SETCOUNTERBUS_API <STD_OFF or STD_ON>
NOTE	

4.4.4 PwmDisableDemReportErrorStatus

Table 4-21. PwmDisableDemReportErrorStatus

Description	Switches the Diagnostics Error Detection and Notification ON or OFF
Class	Non-Autosar Parameter
Range	True, false
Default	False
Source File	Pwm_Cfg.h
Source Representation	#define PWM_DISABLE_DEM_REPORT_ERROR_STATUS<STD_OFF or STD_ON>
NOTE	

4.4.5 MultiPwmChannelSynch

Table 4-22. MultiPwmChannelSynch

Description	This parameter is used to synchronize multiple pwm channels.
Class	Non-Autosar Parameter
Range	True, false
Default	False
Source File	Pwm_Cfg.h
Source Representation	#define MULTI_PWM_CHANNEL_SYNCH<STD_OFF or STD_ON>
NOTE	

4.4.6 PwmSetTriggerDelay

Table 4-23. PwmSetTriggerDelay

Description	Adds / removes the services PwmSetTriggerDelay() from the code.This function is called when the prescaler value needs to be change to maintain same period at different frequency.
Class	Non-Autosar Parameter
Range	True, false
Default	False
Source File	Pwm_Cfg.h
Source Representation	#define PWM_SET_TRIGGER_DELAY_API <STD_OFF or STD_ON>
NOTE	

4.4.7 PwmGetChannelState

Table 4-24. PwmGetChannelState

Description	Switch to indicate that Pwm_GetChannelState API is supported.
Class	Non-Autosar Parameter
Range	True, false
Default	False
Source File	Pwm_Cfg.h
Source Representation	#define PWM_GET_CHANNEL_STATE <STD_OFF or STD_ON>
NOTE	

4.5 PwmChannelConfigSet

This section will describe the hardware specific configuration options supported by the driver are as per AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

4.5.1 PwmChannel

This section describes the channel specific configuration options as described in AUTOSAR PWM Driver software specification Version 4.0 Rev0003.

4.5.1.1 PwmChannelClass

Table 4-25. PwmChannelClass

Description	Class of PWM Channel.
Class	Autosar Parameter
Range	PWM_FIXED_PERIOD, PWM_VARIABLE_PERIOD, PWM_FIXED_PERIOD_SHIFTED
Default	PWM_VARIABLE_PERIOD
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`  /* PwmChannel_0 */`
	`  {`
	`  **PWM_VARIABLE_PERIOD**, /* channel type */`
	`  PWM_HIGH, /* polarity */`
	`  32767, /* default period */`
	`  0x2000, /* default duty */`
	`  PWM_LOW, /* idle state */`
	`  PwmChannel_0_notification, /* notification f() */`
	`  PWM_EMIOS_CHANNEL, /* IP used */`
	`  {`
	`  16, /* EMIOS HW unified channel ID */`
	`  (PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |`
	`  PWM_PRES_1), /* unified channel specific`
	`  parameters */`
	`  0U, /* offset */`
	`  0U, /* trigger delay */`
	`  }`
	`  }`
	`}`

4.5.1.2 PwmChannelId

Table 4-26. PwmChannelId

Description	Channel Id of the PWM channel. This value will be assigned to the symbolic name derived of the PwmChannel container short name.

Table continues on the next page...

Table 4-26. PwmChannelId (continued)

Class	Autosar Parameter
Range	0 - 254
Default	0
Source File	Pwm_Cfg.h

Source Representation	`#define PwmChannel_0 0U`
	`#define PwmChannel_1 1U`
	`#define PwmChannel_2 2U`

4.5.1.3 PwmHwChannel

Table 4-27. PwmHwChannel

Description	Selects the physical PWM Channel.
Class	Implementation Specific Parameter
Range	EMIOS_A_0 to EMIOS_A_31 corresponds to EMIOS_A EMIOS_B_0 to EMIOS_B_31 corresponds to EMIOS_B
Default	EMIOS_A_16
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`  /* PwmChannel_0 */`
	`  {`
	`  PWM_VARIABLE_PERIOD, /* channel type */`
	`  PWM_HIGH, /* polarity */`
	`  32767, /* default period */`
	`  0x2000, /* default duty */`
	`  PWM_LOW, /* idle state */`
	`  PwmChannel_0_notification, /* notification f() */`
	`  PWM_EMIOS_CHANNEL, /* IP used */`
	`  {`
	`  16, /* EMIOS HW unified channel ID */`
	`  (PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |`
	`  PWM_FREEZE_ENABLE | PWM_PRES_1),`
	`  /* unified channel specific`
	`  parameters */`
	`  0U, /* offset */`
	`  0U, /* trigger delay */`
	`  }`
	`  }`
	`}`

4.5.1.4 PwmPolarity

Table 4-28. PwmPolarity

Description	Defines the PWM channel signal polarity.This parameter is used only by the generation code in the transformation from seconds / frequency (if selected) to unified channel ticks.

Table continues on the next page...

Table 4-28. PwmPolarity (continued)

Class	Autosar Parameter
Range	PWM_HIGH, PWM_LOW
Default	PWM_HIGH
Source File	Pwm_Cfg.c, Pwm_PBCfg.c

Source Representation	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`  /* PwmChannel_0 */`
	`  {`
	`  PWM_VARIABLE_PERIOD, /* channel type */`
	`  **PWM_HIGH**, /* polarity */`
	`  32767, /* default period */`
	`  0x2000, /* default duty */`
	`  PWM_LOW, /* idle state */`
	`  PwmChannel_0_notification, /* notification f() */`
	`  PWM_EMIOS_CHANNEL, /* IP used */`
	`  {`
	`  16, /* EMIOS HW unified channel ID */`
	`  (PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |`
	`  PWM_FREEZE_ENABLE | PWM_PRES_1),`
	`  /* unified channel specific`
	`  parameters */`
	`  0U, /* offset */`
	`  0U, /* trigger delay */`
	`  }`
	`  }`
	`}`

4.5.1.5 PwmPeriodDefaultUnits

Table 4-29. PwmPeriodDefaultUnits

Description	Defines the measurement units for PwmPeriodDefault.
Class	Implementation specific parameter
Range	Period_in_seconds / Period_in_ticks / Frequency_in_Hz
Default	Period_in_seconds
Source File	This parameter is used only by the generation code in the transformation from seconds / frequency (if selected) to unified channel ticks.

Source Representation	`The highlighted portion of the following structure:`
	
	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	
	`{`
	` PWM_VARIABLE_PERIOD, /* channel type */`
	` PWM_HIGH, /* polarity */`
	` **32767**, /* default period in ticks after`
	`transformation from seconds or Hz if selected */`
	` 0x2000, /* default duty */`
	` PWM_LOW, /* idle state */`
	` PwmChannel_0_notification, /* notification f() */`
	
	` PWM_EMIOS_CHANNEL, /* IP used */`

Table 4-29. PwmPeriodDefaultUnits

	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | PWM_PRES_1),
	/* unified channel specific parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	},
	
	}

4.5.1.6 PwmPeriodDefault

Table 4-30. PwmPeriodDefault

Description	Value of period used for Initialization. The Unit of Period is defined by PwmPeriodDefaultUnits. Note: Counter will always start from 0x1, so the period value is incremented by 1 in the code, then the value is updated into period register. Valid values [1, 0xFFFE = 65534] In case of PWM_MODE_OPWMB or PWM_MODE_OPWMT this value is ignored and the period for this channel is equal with the period of the selected unified channel running in MCB mode. Please note that the period of the MCB channel must be less than 0xFFFF or else reaching 0% will not be possible.
Class	Autosar Parameter
Range	1 - 65534
Default	32767
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	CONST(Pwm_ChannelConfigType, PWM_CONST)
	Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =
	{
	/* PwmChannel_0 */
	{
	PWM_VARIABLE_PERIOD, /* channel type */
	PWM_HIGH, /* polarity */
	32767, /* default period */
	0x2000, /* default duty */
	PWM_LOW, /* idle state */
	PwmChannel_0_notification, /* notification f() */
	PWM_EMIOS_CHANNEL, /* IP used */
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |
	PWM_FREEZE_ENABLE | PWM_PRES_1),
	/* unified channel specific
	parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	
	}
	}

4.5.1.7 PwmDutyCycleDefault

Table 4-31. PwmDutyCycleDefault

Description	Value of duty cycle used for initialization 0x0 represents 0% 0x8000(32768) represents 100%
Class	Implementation Specific Parameter
Range	0 - 0x8000 (32768)
Default	16384 (50%)
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`  /* PwmChannel_0 */`
	`  {`
	`  PWM_VARIABLE_PERIOD, /* channel type */`
	`  PWM_HIGH, /* polarity */`
	`  32767, /* default period */`
	`  **0x2000**, /* default duty */`
	`  PWM_LOW, /* idle state */`
	`  PwmChannel_0_notification, /* notification f() */`
	`  PWM_EMIOS_CHANNEL, /* IP used */`
	`  {`
	`  16, /* EMIOS HW unified channel ID */`
	`  (PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |`
	`  PWM_PRES_1), /* unified channel specific`
	`  parameters */`
	`  0U, /* offset */`
	`  0U, /* trigger delay */`
	`  }`
	`  }`
	`}`

4.5.1.8 PwmIdleState

Table 4-32. PwmIdleState

Description	Value of the PWM signal at idle state.
Class	Autosar Parameter
Range	PWM_LOW, PWM_HIGH
Default	PWM_LOW
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`  /* PwmChannel_0 */`
	`  {`
	`  PWM_VARIABLE_PERIOD, /* channel type */`
	`  PWM_HIGH, /* polarity */`
	`  32767, /* default period */`
	`  0x2000, /* default duty */`
	`  **PWM_LOW**, /* idle state */`
	`  PwmChannel_0_notification, /* notification f() */`

Table 4-32. PwmIdleState

	PWM_EMIOS_CHANNEL, /* IP used */
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |
	PWM_FREEZE_ENABLE | PWM_PRES_1),
	/* unified channel specific
	parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	
	}
	}

4.5.1.9 PwmNotification

Table 4-33. PwmNotification

Description	User notification callback function.
Class	Autosar Parameter
Range	N/A
Default	NULL_PTR
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	CONST(Pwm_ChannelConfigType, PWM_CONST)
	Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =
	{
	/* PwmChannel_0 */
	{
	PWM_VARIABLE_PERIOD, /* channel type */
	PWM_HIGH, /* polarity */
	32767, /* default period */
	0x2000, /* default duty */
	PWM_LOW, /* idle state */
	PwmChannel_0_notification, /* notification f() */
	PWM_EMIOS_CHANNEL, /* IP used */
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |
	PWM_FREEZE_ENABLE | PWM_PRES_1),
	/* unified channel specific
	parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	
	}
	}

4.5.1.10 PwmModeSelect

Table 4-34. PwmModeSelect

Description	Select the PwmMode for the channel from drop downlist. OPWFMB, OPWMB, OPWMT, DAOC,OPWMCB_LEAD_DEADTIME, OPWMCB_TRAIL_DEADTIME. Variable period channels are implemented using only the OPWFMB and DAOC mode. The plug-in will check this and raise an error if this condition is not satisfied.
Class	Implementation Specific Parameter
Range	PWM_MODE_OPWFMB, PWM_MODE_OPWMB,PWM_MODE_DAOC,PWM_MODE_OPWMT, PWM_MODE_OPWMCB_LEAD_DEADTIME, PWM_MODE_OPWMCB_TRAIL_DEADTIME
Default	PWM_MODE_OPWFMB
Source File	Pwm_Cfg.c, Pwm_PBCfg.c

Source Representation	The highlighted portion of the following structure:
	
	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	
	`{`
	`PWM_VARIABLE_PERIOD, /* channel type */`
	`PWM_HIGH, /* polarity */`
	`32767, /* default period */`
	`0x2000, /* default duty */`
	`PWM_LOW, /* idle state */`
	`PwmChannel_0_notification, /* notification f() */`
	
	`PWM_EMIOS_CHANNEL, /* IP used */`
	
	`{`
	`16, /* EMIOS HW unified channel ID */`
	`(PWM_BUS_INTERNAL_COUNTER | **PWM_MODE_OPWFMB** | PWM_PRES_1),`
	`/* unified channel specific parameters */`
	`0U, /* offset */`
	`0U, /* trigger delay */`
	`}`
	`},`
	
	`}`

4.5.1.11 EmiosUnifiedChannelBusSelect

Table 4-35. EmiosUnifiedChannelBusSelect

Description	Selects the counter used with the unified channel InternalCounter – mandatory for OPWFMB BusA, BusDiverse – mandatory for OPWMB/ OPWMT
Class	Implementation Specific Parameter
Range	PWM_BUS_A,PWM_BUS_DIVERSE,PWM_BUS_INTERNAL_COUNTER
Default	PWM_BUS_INTERNAL_COUNTER
Source File	Pwm_Cfg.c, Pwm_PBCfg.c
Source Representation	The highlighted portion of the following structure:

Table continues on the next page...

Table 4-35. EmiosUnifiedChannelBusSelect (continued)

	CONST(Pwm_ChannelConfigType, PWM_CONST)
	Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =
	{
	
	{
	PWM_VARIABLE_PERIOD, /* channel type */
	PWM_HIGH, /* polarity */
	32767, /* default period */
	0x2000, /* default duty */
	PWM_LOW, /* idle state */
	PwmChannel_0_notification, /* notification f() */
	
	PWM_EMIOS_CHANNEL, /* IP used */
	
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | PWM_PRES_1),
	/* unified channel specific parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	},
	
	}

Note	When PWM_BUS_DIVERSE or PWM_BUS_A is selected for "EmiosUnifiedChannelBusSelect", this channels (Counter Bus A/B/C/D/E) must have a lower "PwmChannelId" as for the other channels which are using this bus counter. Otherwise the PWM_BUS_DIVERSE or PWM_BUS_A will be not initialized during the intitialization of the channels which are using this bus counters.

4.5.1.12 PwmPrescaler

Table 4-36. PwmPrescaler

Description	Pwm Channel Prescaler. Clock prescaler to decrease Pwm period. Affects the PWM period only in OPWFMB mode when the internal counter must be selected.
Class	Implementation Specific Parameter
Range	PwmPrescalerDiv1 To PwmPrescalerDiv4
Default	PwmPrescalerDiv1
Source File	Pwm_Cfg.c, Pwm_PBCfg.c
Source Representation	CONST(Pwm_ChannelConfigType, PWM_CONST) Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] = { { PWM_VARIABLE_PERIOD, /* channel type */ PWM_HIGH, /* polarity */ 32767, /* default period */ 0x2000, /* default duty */ PWM_LOW, /* idle state */ PwmChannel_0_notification, /* notification f() */ PWM_EMIOS_CHANNEL, /* IP used */ {

Table 4-36. PwmPrescaler

	16, /* EMIOS HW unified channel ID */ (PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | **PWM_PRES_1**), /* unified channel specific parameters */ 0U, /* offset */ 0U, /* trigger delay */ } }, }

4.5.1.13 PwmPrescaler_Alternate

Table 4-37. PwmPrescaler_Alternate

Description	Alternate Pwm Channel Prescaler.
Class	Non-Autosar Parameter
Range	PwmPrescalerDiv1 To PwmPrescalerDiv4
Default	PwmPrescalerDiv1
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	{
	PWM_VARIABLE_PERIOD, /* channel type */
	PWM_HIGH, /* polarity */
	32767, /* default period */
	0x2000, /* default duty */
	PWM_LOW, /* idle state */
	PwmChannel_0_notification, /* notification f() */
	PWM_EMIOS_CHANNEL, /* IP used */
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB |
	PWM_FREEZE_ENABLE | **PWM_PRES_1**),
	/* unified channel specific
	parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	
	}

4.5.1.14 PwmOffset

Table 4-38. PwmOffset

Description	PwmOffset (in ticks) of the PWM output. This parameter is used only in OPWMB/OPWMT modes. Counter will always start from 1, So the offset value is incremented by 1 in the code, then the value is updated into register. Please note the PwmOffset parameter must be less than the period of the MCB channel used as reference. When Offset !=0 100% dutycycle can't be reached due to the fact that the PWM signal is shifted offset ticks. The max dutycycle achievable when PwmOffset != 0 can be calculated using the following formula. Dutycycle < ((Period - PwmOffset) / Period) * 0x8000. Period is the period in ticks of the MCB channel

Table continues on the next page...

Table 4-38. PwmOffset (continued)

	used as reference. If this condition is not satisfied then Det_ReportError(PWM_MODULE_ID, 0, PWM_SETDUTYCYCLE_ID, PWM_E_OPWMB_CHANNEL_OFFSET_DUTYCYCLE_RANGE) will be generated.
Class	Implementation Specific Parameter
Range	(0 – MCB channel period) – please see limitation formula for max dutycycle
Default	0
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	The highlighted portion of the following structure:
	
	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	`{`
	`PWM_VARIABLE_PERIOD, /* channel type */`
	`PWM_HIGH, /* polarity */`
	`32767, /* default period */`
	`0x2000, /* default duty */`
	`PWM_LOW, /* idle state */`
	`PwmChannel_0_notification, /* notification f() */`
	`PWM_EMIOS_CHANNEL, /* IP used */`
	`{`
	`16, /* EMIOS HW unified channel ID */`
	`(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | PWM_PRES_1),`
	`/* unified channel specific parameters */`
	`**0U**, /* offset */`
	`0U, /* trigger delay */`
	`}`
	`},`
	`}`

4.5.1.15 PwmTriggerDelay

Table 4-39. PwmTriggerDelay

Description	PwmTriggerDelay specifies the position of the trigger. Please note that if the configured value is outside of the configured period (reference channel period), the trigger will never be generated.
Class	Implementation Specific Parameter
Range	[1-max period of reference channel]
Default	0
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	The highlighted portion of the following structure:
	
	`CONST(Pwm_ChannelConfigType, PWM_CONST)`
	`Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =`
	`{`
	
	`{`
	`PWM_VARIABLE_PERIOD, /* channel type */`
	`PWM_HIGH, /* polarity */`
	`32767, /* default period */`
	`0x2000, /* default duty */`
	`PWM_LOW, /* idle state */`
	`PwmChannel_0_notification, /* notification f() */`

Table 4-39. PwmTriggerDelay

	PWM_EMIOS_CHANNEL, /* IP used */
	
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | PWM_PRES_1),
	/* unified channel specific parameters */
	0U, /* offset */
	123U, /* trigger delay */
	}
	},
	}

4.5.1.16 PwmFreezeEnable

Table 4-40. PwmFreezeEnable

Description	Freeze Enable Bit. The FREN bit, if set and validated by FRZ bit in EMIOS_MCR register, freezes all unified channel registers when in debug mode to allow the MCU to perform debug functions.
Class	Implementation Specific Parameter
Range	True, false
False	NULL_PTR
Source File	Pwm_Cfg.c, Pwm_PBcfg.c

Source Representation	The highlighted portion of the following structure:
	
	CONST(Pwm_ChannelConfigType, PWM_CONST)
	Pwm_InitChannel_0[PWM_CONF_CHANNELS_PB_0] =
	{
	
	{
	PWM_VARIABLE_PERIOD, /* channel type */
	PWM_HIGH, /* polarity */
	32767, /* default period */
	0x2000, /* default duty */
	PWM_LOW, /* idle state */
	PwmChannel_0_notification, /* notification f() */
	
	PWM_EMIOS_CHANNEL, /* IP used */
	
	{
	16, /* EMIOS HW unified channel ID */
	(PWM_BUS_INTERNAL_COUNTER | PWM_MODE_OPWFMB | PWM_FREEZE_ENABLE | PWM_PRES_1),
	/* unified channel specific parameters */
	0U, /* offset */
	0U, /* trigger delay */
	}
	},
	
	}

How to Reach Us:

Home Page:
freescale.com

Web Support:
freescale.com/support

Document Number UM13PWMASR4.0R1.0.1
Revision 1.2

AUTOSAR

freescale™